



COMP7308: Introduction to Unmanned Systems

School of Computing and Data Science

Department of AI and Data Science

University of Hong Kong

Advanced Topic of Particle Filter

Jia PAN

Advanced Topic 4

Particle filter algorithm

- For each particle in S_{t-1}
 - Propagate sample forward using motion model (sampling)
$$\bar{x}_t^{[i]} \sim p(x_t \mid x_{t-1}^{[i]}, u_t)$$
 - Calculate the importance weight
$$w_t^{[i]} = p(z_t \mid \bar{x}_t^{[i]})$$
 - Store in an interim particle set
$$S'_t = S'_t \cup (\bar{x}_t^{[i]}, w_t^{[i]})$$
- Normalize weights
- For $j = 1$ to I
 - Draw index i with probability proportional to $w_t^{[i]}$ (resampling)
 - Add to final particle set $S_t = S_t \cup (x_t^{[j]}, 1)$

Particle filter derivation

- Consider the particles as state sequence samples

$$x_{0:t}^{[i]} = x_0^{[i]}, x_1^{[i]}, \dots, x_t^{[i]}$$

- Form belief over the entire sequence

$$bel(x_{0:t}) = p(x_{0:t} \mid u_{1:t}, z_{1:t})$$

– Instead of just

$$bel(x_t) = p(x_t \mid u_{1:t}, z_{1:t})$$

- This is an enormous state to approximate with a set of particles; but no matter, for derivation only

Particle filter derivation

- Using Bayes theorem, expand belief about the last measurement

$$\begin{aligned} \text{bel}(x_{0:t}) &= p(x_{0:t} \mid u_{1:t}, z_{1:t}) = p(\overset{\textcircled{\text{a}}}{x_{0:t}} \mid u_{1:t}, \overset{\textcircled{\text{c}}}{z_{1:t-1}}, \overset{\textcircled{\text{b}}}{z_t}) \quad p(a|b, \textcolor{red}{c}) = \frac{p(b|a, \textcolor{red}{c})p(a \mid \textcolor{red}{c})}{p(b \mid \textcolor{red}{c})} \\ &= \frac{p(z_t \mid x_{0:t}, u_{1:t}, z_{1:t-1})p(x_{0:t} \mid u_{1:t}, z_{1:t-1})}{p(z_t \mid u_{1:t}, z_{1:t-1})} \\ &= \eta \cdot p(z_t \mid x_{0:t}, u_{1:t}, z_{1:t-1})p(x_{0:t} \mid u_{1:t}, z_{1:t-1}) \end{aligned}$$

- The Markov assumption remains valid, thus

$$= \eta \cdot p(z_t \mid x_t)p(x_{0:t} \mid u_{1:t}, z_{1:t-1})$$

Particle filter derivation

- Conditional probability can be used to expand the last distribution

$$\begin{aligned} bel(x_{0:t}) &= \eta \cdot p(z_t \mid x_t) p(x_{0:t} \mid u_{1:t}, z_{1:t-1}) && \boxed{p(a, b \mid c) = p(a \mid b, c) p(b \mid c)} \\ &\quad \textcircled{a} \quad \textcircled{b} \quad \textcircled{c} \\ &= \eta \cdot p(z_t \mid x_t) p(x_t, x_{0:t-1} \mid u_{1:t}, z_{1:t-1}) \\ &= \eta \cdot p(z_t \mid x_t) p(x_t \mid x_{0:t-1}, u_{1:t}, z_{1:t-1}) p(x_{0:t-1} \mid u_{1:t}, z_{1:t-1}) \end{aligned}$$

- Apply the Markov assumption again yields

$$= \eta \cdot p(z_t \mid x_t) p(x_t \mid x_{t-1}, u_t) p(x_{0:t-1} \mid u_{1:t}, z_{1:t-1})$$

Particle filter derivation

- The sequence $x_{0:t-1}$ does not depend on u_t

$$\begin{aligned}bel(x_{0:t}) &= \eta \cdot p(z_t \mid x_t)p(x_t \mid x_{t-1}, u_t)p(x_{0:t-1} \mid u_{1:t}, z_{1:t-1}) \\&= \eta \cdot p(z_t \mid x_t)p(x_t \mid x_{t-1}, u_t)p(x_{0:t-1} \mid u_{1:t-1}, z_{1:t-1}) \\&= \eta \cdot p(z_t \mid x_t)p(x_t \mid x_{t-1}, u_t)bel(x_{0:t-1})\end{aligned}$$

- Breaking into two steps

- Prediction:

$$\overline{bel}(x_{0:t}) = p(x_t \mid x_{t-1}, u_t)bel(x_{0:t-1})$$

- The i -th particle generated by this distribution is an element of the predicted belief particle set

Particle filter derivation

- The measurement update uses the importance sampling to generate a particle set representation of belief

- Weighting, based on the relation to the predicted belief

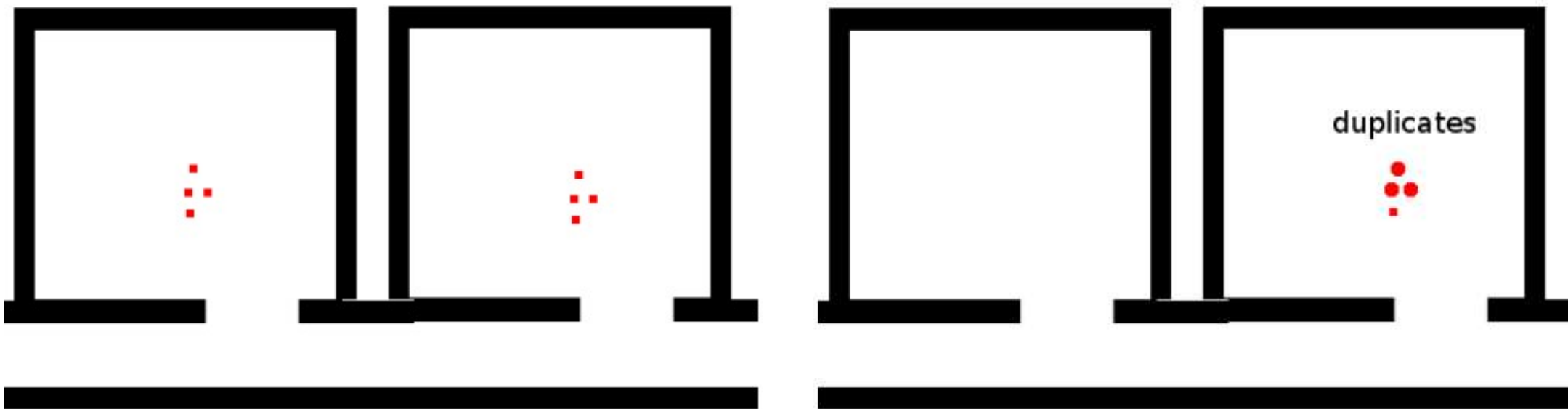
$$\begin{aligned}w_t^{[i]} &= \eta \cdot \frac{bel(x_{0:t})}{\overline{bel}(x_{0:t})} \\&= \eta \cdot \frac{p(z_t | x_t)p(x_t | x_{t-1}, u_t)bel(x_{0:t-1})}{p(x_t | x_{t-1}, u_t)bel(x_{0:t-1})} \\&= \eta \cdot p(z_t | x_t)\end{aligned}$$

- Which confirms the use of measurement model as weighting parameter
- This confirms that the particle sets are distributed according to the full belief sequences, which must also hold for the current state too.

Problems of particle filter in practice and solutions

Problem 1: two-room challenge

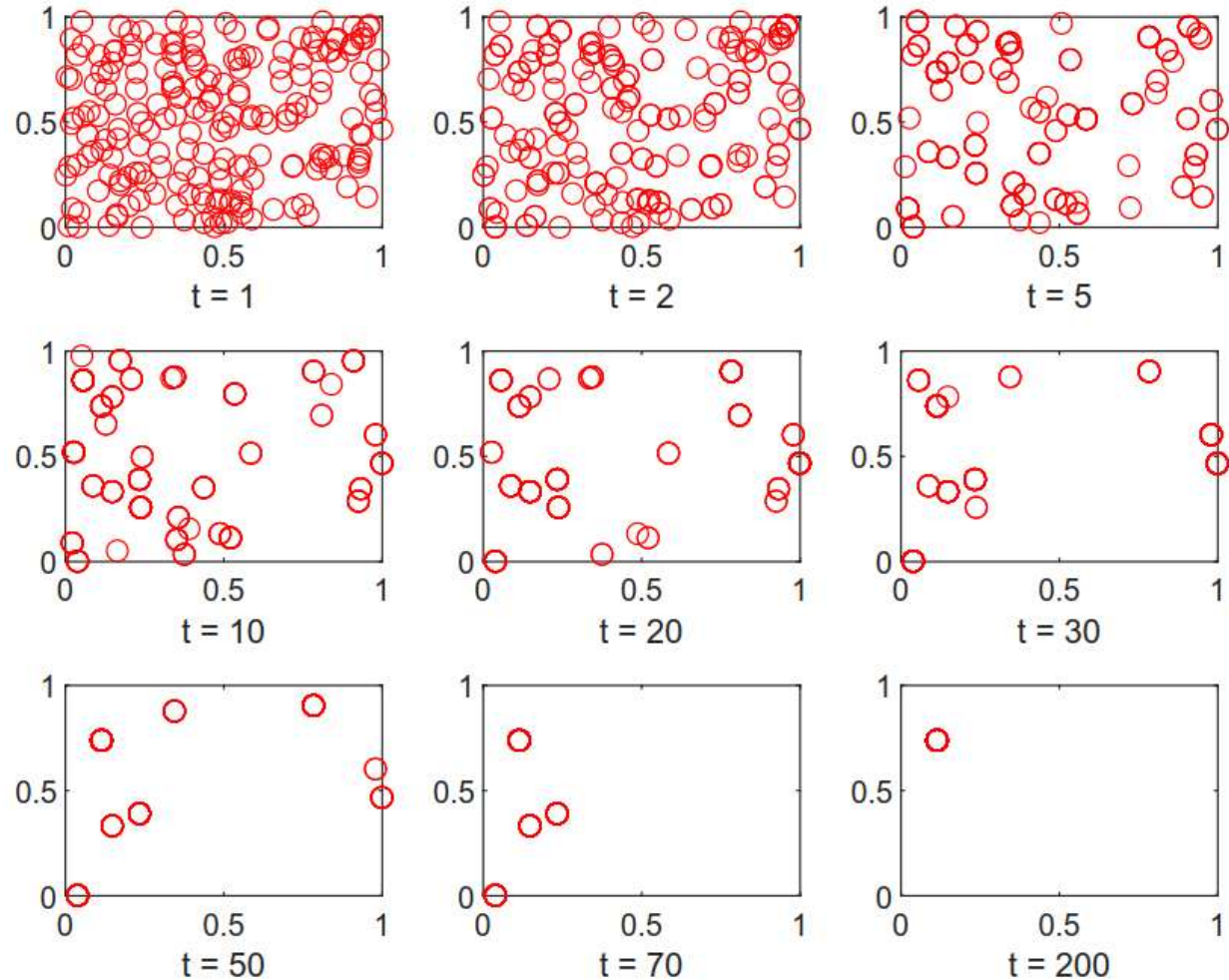
- Given: particles equally distributed, no motion, no observation
- What happens?



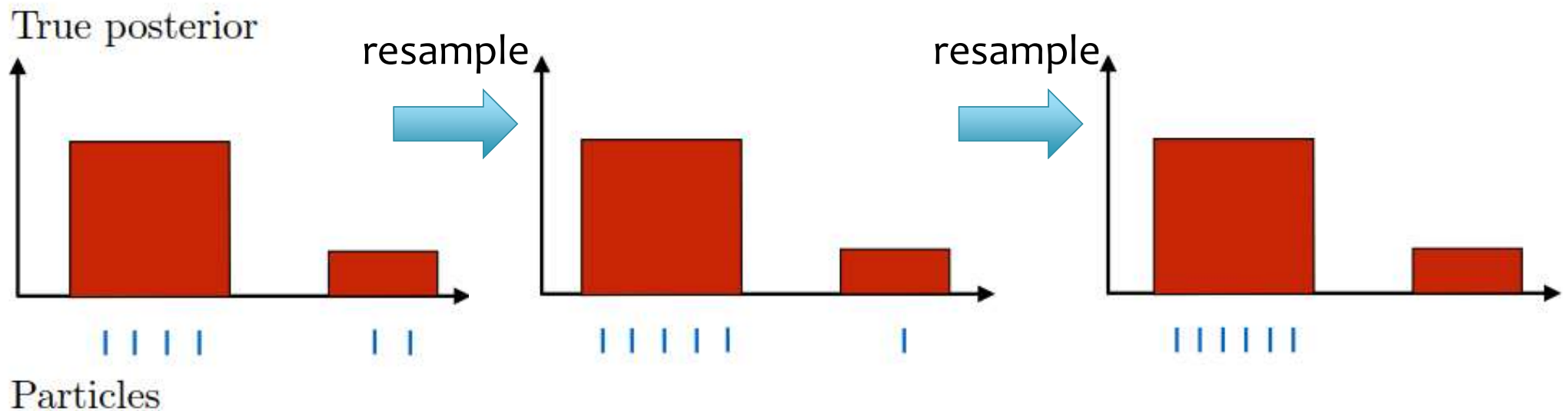
All particles migrate to one of two rooms!!

Similar extreme case

- No motion: $x_t = x_{t-1}$
- No measurements, uniform weights on each particle
- Uniform prior over 2D space



Reason: resampling increases variance w.r.t. true posterior $E[[f(x) - g(x)]^2]$

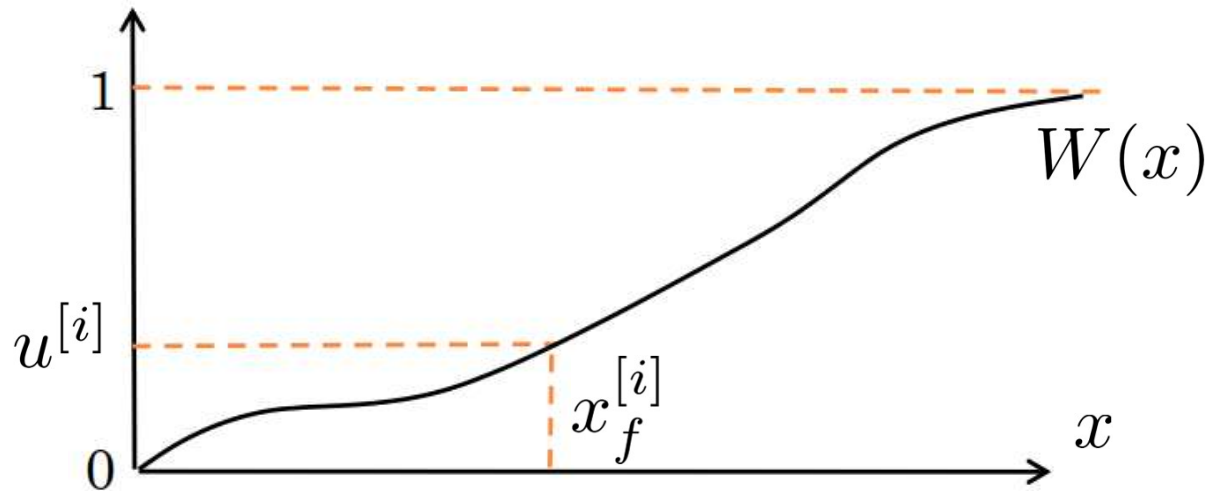


- Resampling is correct in mean (non-bias) $E[f(x) - g(x)] = 0$ but
 - collapses particles
 - reduces diversity
 - increases variance w.r.t. true posterior

Fix 1: choose when to resample

- **Key idea:** if variance of weights is low, don't resample
- We can implement this idea in various ways:
 1. All weights are equal – don't resample
 2. Entropy of weights is high – don't resample
 3. Ratio of max to min weights is low – don't resample

Fix 2: low variance sampling



$$W(x_1) \approx w(x_1)$$

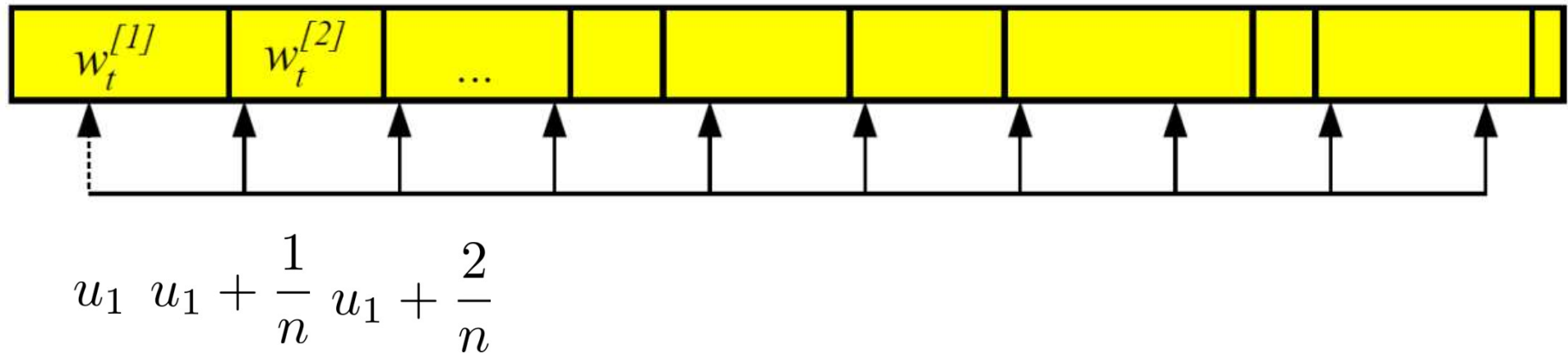
$$W(x_2) \approx w(x_1) + w(x_2)$$

$$W(x_3) \approx w(x_1) + w(x_2) + w(x_3)$$

$$W(x_4) \approx w(x_1) + w(x_2) + w(x_3) + w(x_4)$$

How many
random samples
we need when
using particle
filtering?

Fix 2: low variance sampling

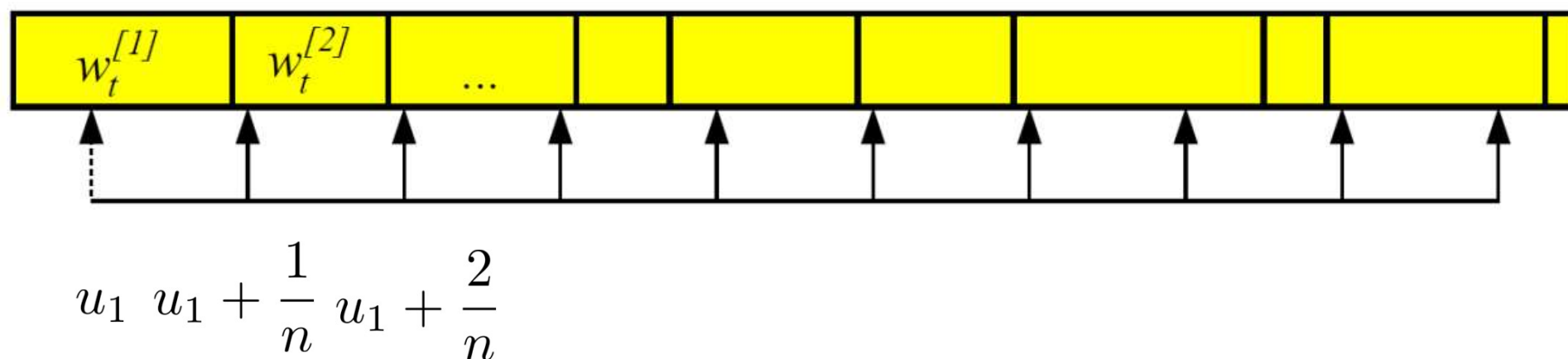


- Reduce randomness $\rightarrow$ low variance
- Linear time complexity
- Easy to implement

Low variance sampling

1. Algorithm **systematic_resampling**(S, n):
2. $S' = \emptyset, c_1 = w^1$
3. **For** $i = 2 \dots n$ *Generate cdf*
4. $c_i = c_{i-1} + w^i$
5. $u_1 \sim U[0, n^{-1}], i = 1$ *Initialize threshold*
6. **For** $j = 1 \dots n$ *Draw samples ...*
7. **While** ($u_j > c_i$) *Skip until next threshold reached*
8. $i = i + 1$
9. $S' = S' \cup \{ < x^i, n^{-1} > \}$ *Insert*
10. $u_{j+1} = u_j + n^{-1}$ *Increment threshold*
11. **Return** S'

Why does this work?



- What happens when all weights equal?
- What happens if you have ONE large weight and many tiny weights?

$$w_1 = 0.5, w_2 = 0.5/1000, w_3 = 0.5/1000, \dots, w_{1001} = 0.5/1000$$

Problem 2: particle starvation

- No particle in the vicinity of the current state

- Why?

1. Unlucky set of samples

2. Committed to the wrong mode in a multi-modal scenario

3. Bad set of sensor measurements



Fix: add new particles

- Which distribution should be used to add new particles?
 1. Uniform distribution
 2. Biased around the last good measurement
 3. Directly from the sensor model

Fix: add new particles

- When should we add new samples?
 - Key idea: as soon as importance weights become too small, add more samples
1. Threshold the total sum of weights
 2. Fancy estimator that checks rate of change

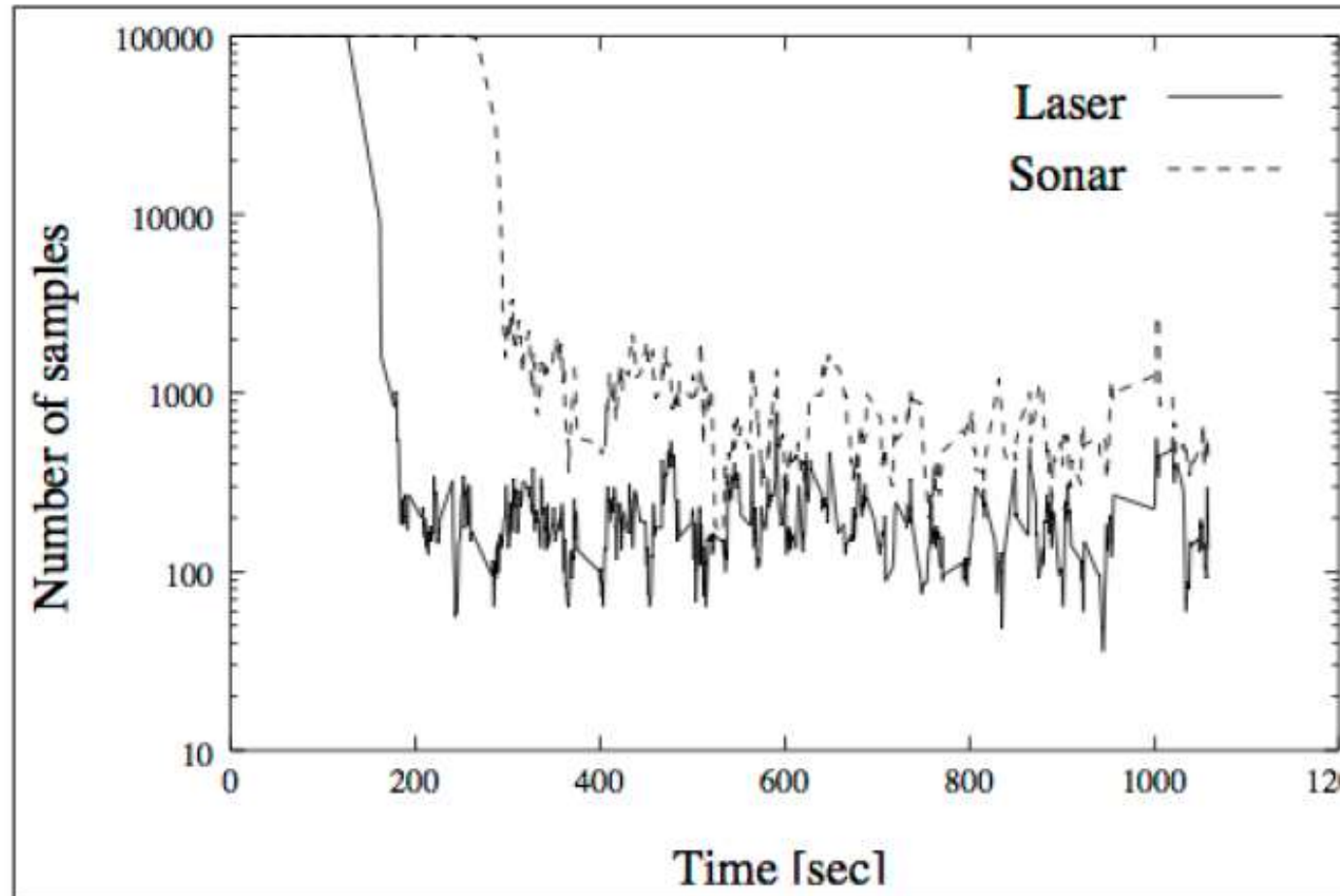
Problem 3: observation model not good

- Consider a measurement obtained with a **noise-free** sensor, what will happen?
 - All particles would end up with weight zero, as it is very unlikely to have had a particles matching the measurement exactly.
- Fixes
 1. Samples from a better proposal distribution than the motion model
 2. “smooth” out the observation model (apply a power of $1/m$ to all probabilities; or m observations count as one)
 3. Last resort: smooth your observation model with a Gaussian (you are pretending your observation model is worse than it is)

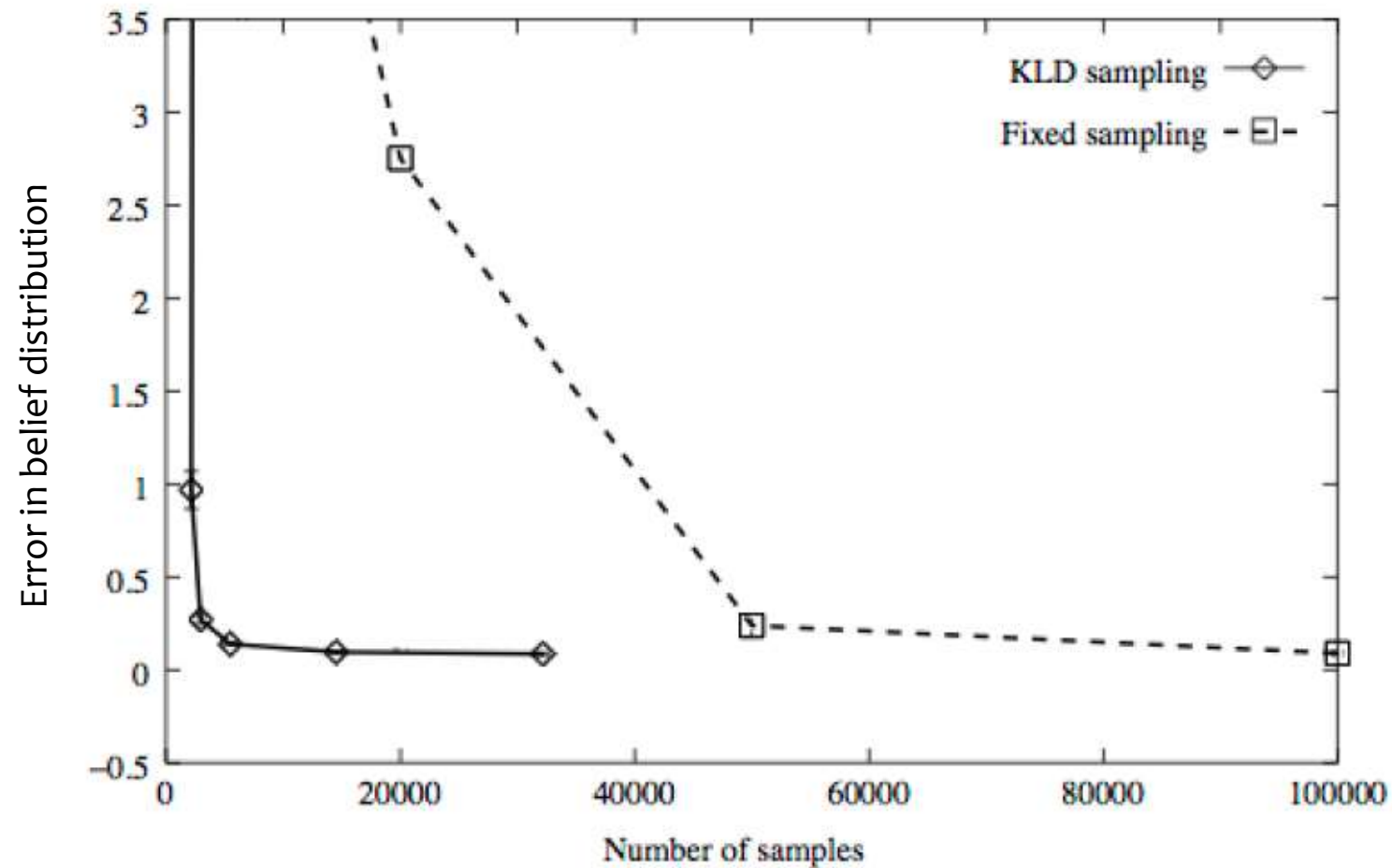
Problem 4: How many samples is enough?

- Example: we typically need **more particles at the beginning** of run
- Key idea (KLD sampling): the number of samples we need depends on how scattered the samples are.
 - Partition the state-space into bins (e.g. by clustering)
 - When sampling, keep track of the number of bins
 - Stop sampling when you reach a statistical threshold that depends on the number of bins
 - if all samples fall in a small number of bins $\Rightarrow$ lower threshold

KLD sampling



KLD sampling vs fixed number sampling



Problem 5: better proposal?

- Target distribution

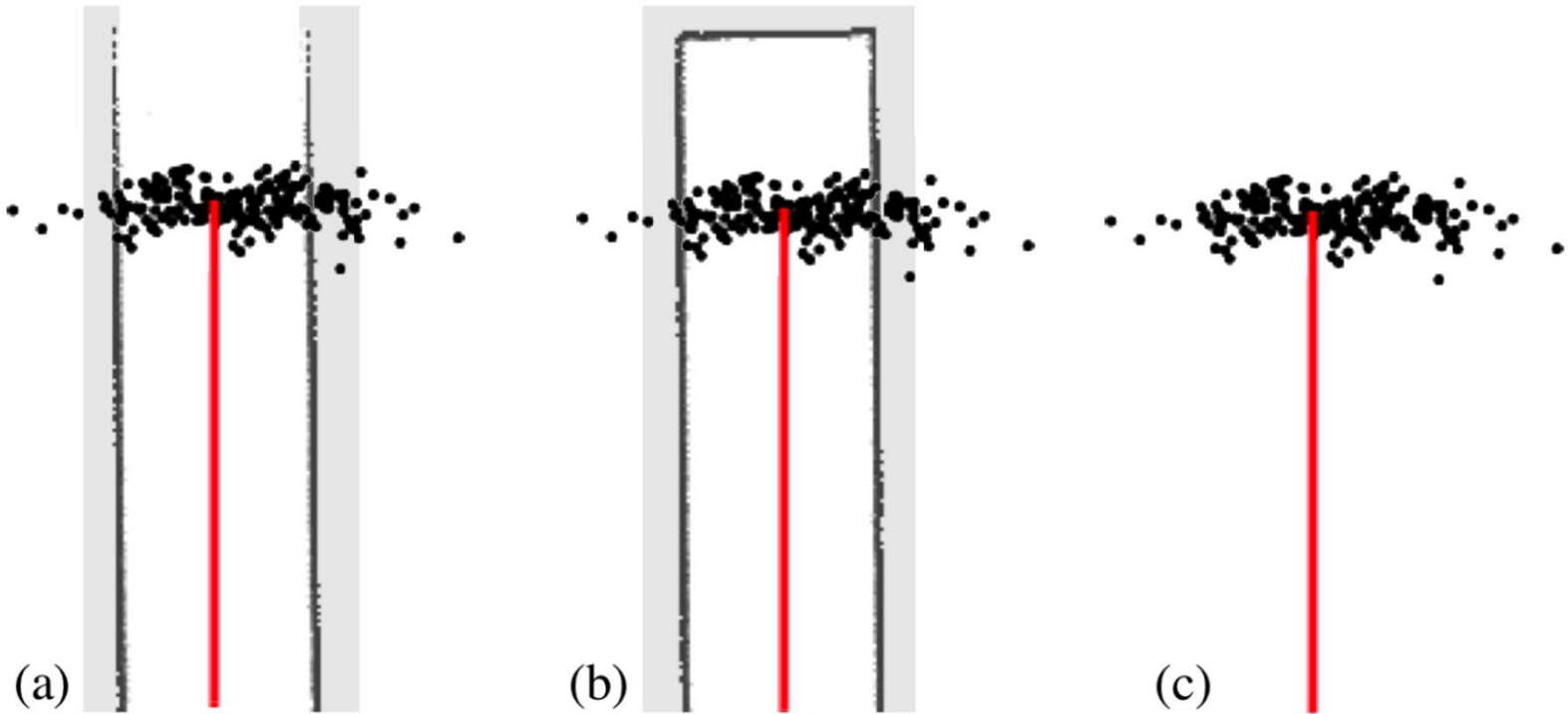
$$bel(x_t) = \eta p(\textcolor{red}{z}_t \mid x_t) \int p(x_t \mid x_{t-1}, \textcolor{blue}{u}_t) bel(x_{t-1}) dx_{t-1}$$

- Proposal distribution

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, \textcolor{blue}{u}_t) bel(x_{t-1}) dx_{t-1}$$

Question

- Given the same $bel(x_{t-1})$ what are the resulting proposal distributions for all cases?



Different map, different proposal

